

Sample efficiency and training speed for G1 locomotion

A controlled comparison of RSL PPO, Warp PPO, and the authors' FlashSAC implementation on the same Isaac Lab task.

Maximilian Krause · 9 October 2026 · Three independent learner seeds · One isolated NVIDIA L40 per run · 50.38 million transitions per run

Abstract

FlashSAC reached the measured walking criterion with fewer collected transitions and achieved lower final velocity error in this cohort. Warp PPO reduced PPO's learning-phase cost by 18.6%, but rollout dominated runtime, limiting its training-loop gain to 2.5%. All nine final policies passed the forward-walking criterion. Preparation and evaluation costs make the complete command-pipeline comparison less favorable to Warp and FlashSAC than the training-loop numbers alone suggest.

2 of 3 seeds

FlashSAC passed at 12.29M transitions. Neither PPO had a passing seed at that checkpoint. This is evidence of better observed sample efficiency.

1.23× faster updates

Warp PPO: 125.5 seconds of learning versus RSL PPO's 154.2 seconds. The complete production training loop was only 1.025× faster.

100% forward survival

All learners survived every final forward evaluation. Native-command survival was 87.5% for RSL, 99.5% for Warp, and 100% for FlashSAC.

01 How much experience was needed?

At 24.58M transitions, all three FlashSAC seeds passed the walking gate, versus two seeds for each PPO. The first passing checkpoints were 500 / 500 / 1000 iterations for FlashSAC, 1000 / 1500 / 1000 for Warp PPO, and 1000 / 1000 / 2050 for RSL PPO. One RSL seed later failed at iteration 1500 before recovering.

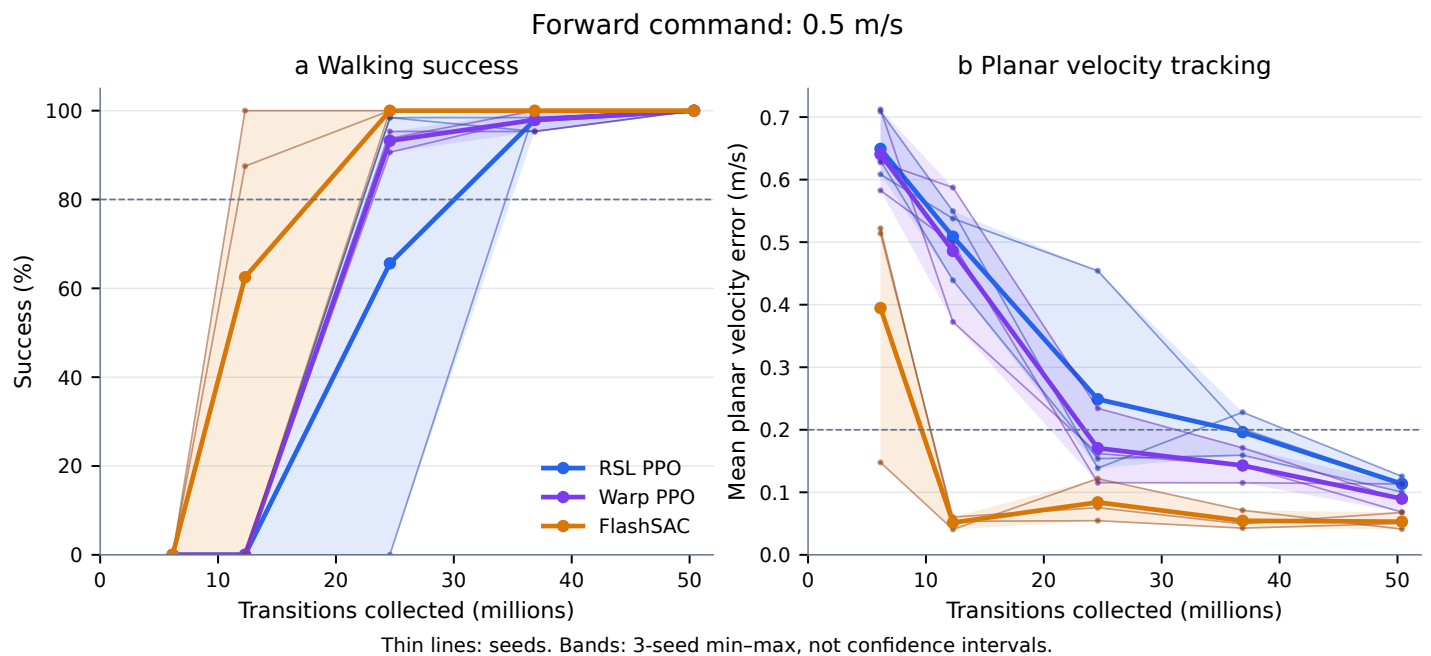


Figure 1. Forward 0.5 m/s evaluations. Dark thin lines show seeds; colored lines are means and bands are three-seed min-max ranges, not confidence intervals. Lines connect five measured checkpoints; intermediate values were not evaluated. Episode walking success requires survival, planar error below 0.25 m/s, yaw error below 0.4 rad/s and forward speed above 0.25 m/s. Dashed 80% success and 0.2 m/s mean-error thresholds are two aggregate gate conditions; survival of at least 90% is also required.

Reading the curves. Thin lines show individual training seeds; the thick line is their mean and the band is their minimum-maximum range. The band is not a confidence interval. Each point comes from 64 evaluation worlds. Lines connect sampled checkpoints; behavior between them was not measured.

► Per-seed passing checkpoints and observed regressions

02 Where did the time go?

PPO spent approximately 82–85% of its training loop collecting rollouts. Accelerating its remaining learning phase therefore produced a small overall gain. FlashSAC performed more, smaller replay update calls and used larger networks; its higher full-budget runtime coexists with earlier measured learning success.

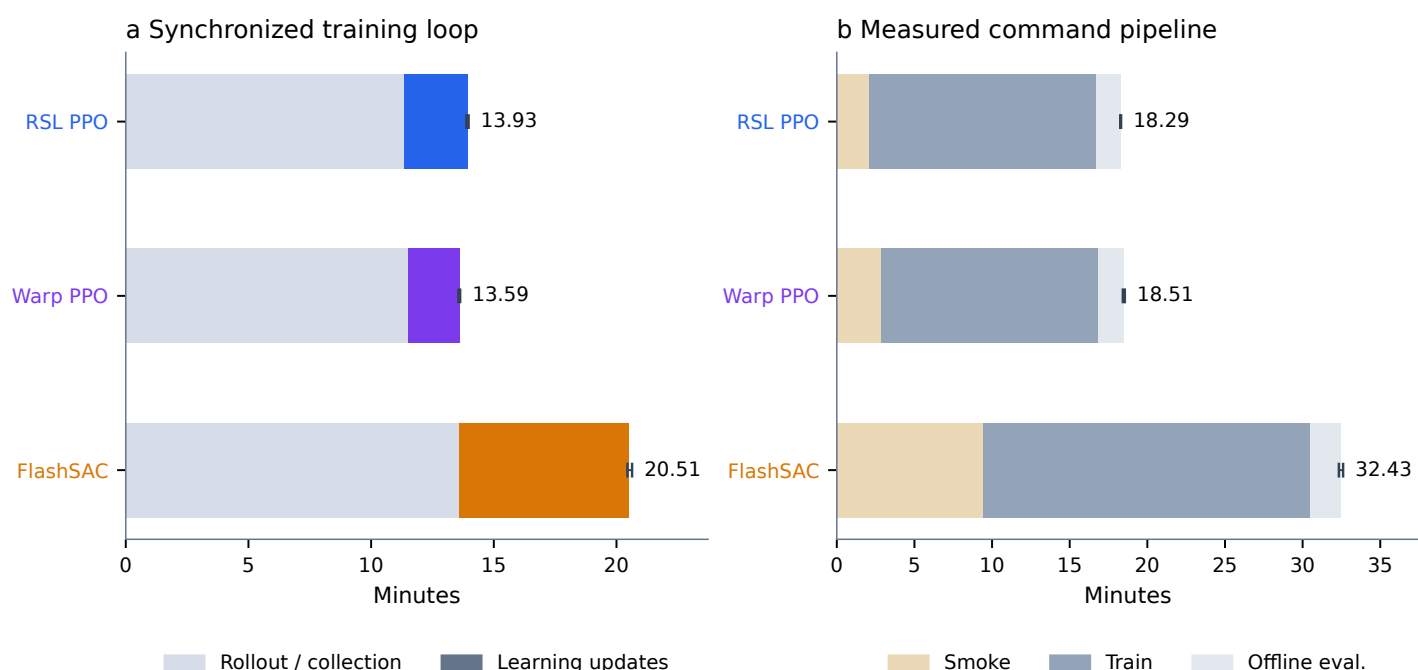


Figure 2. Means and total-time min-max whiskers over three seeds. Left: synchronized training-loop phases. Flash interleaves learning and attributes updates using CUDA events; host launch overhead remains in the collection remainder. Algorithm-specific learning work differs. Right: measured smoke, training and offline-evaluation command costs. These are not fully cold timings; OSMO queue, checkout/bootstrap and artifact retrieval are excluded.

Learner	Training loop min ↓	Learning phase min ↓	Preceding smoke min ↓	Smoke + train + evaluate min ↓
RSL PPO	13.93	2.57	2.05	18.29
Warp PPO	13.59	2.09	2.83	18.51
FlashSAC	20.51	6.91	9.43	32.43

Three-seed means. The complete measured command pipeline took 18.29 / 18.51 / 32.43 minutes for RSL / Warp / FlashSAC. It excludes OSMO queueing, container bootstrap, and artifact retrieval. This is not a cache-cleared machine-start benchmark.

What “faster” means here. Warp’s production loop is faster; the measured command pipeline, including its mandatory preparation smoke and evaluations, is slightly slower than RSL’s. Flash compilation also enters its early production iterations. Flash’s rollout/update split uses CUDA-event attribution, whereas PPO phases use synchronized wall timing; these phase totals do not compare identical gradient work.

03 What did the final policies achieve?

At the matched final budget, FlashSAC had the lowest mean planar velocity error in both evaluation scenarios. Every forward evaluation passed walking and survival. Native commands are broader: they include varying planar and yaw commands as well as standing, so native tracking success is a different metric from forward walking.

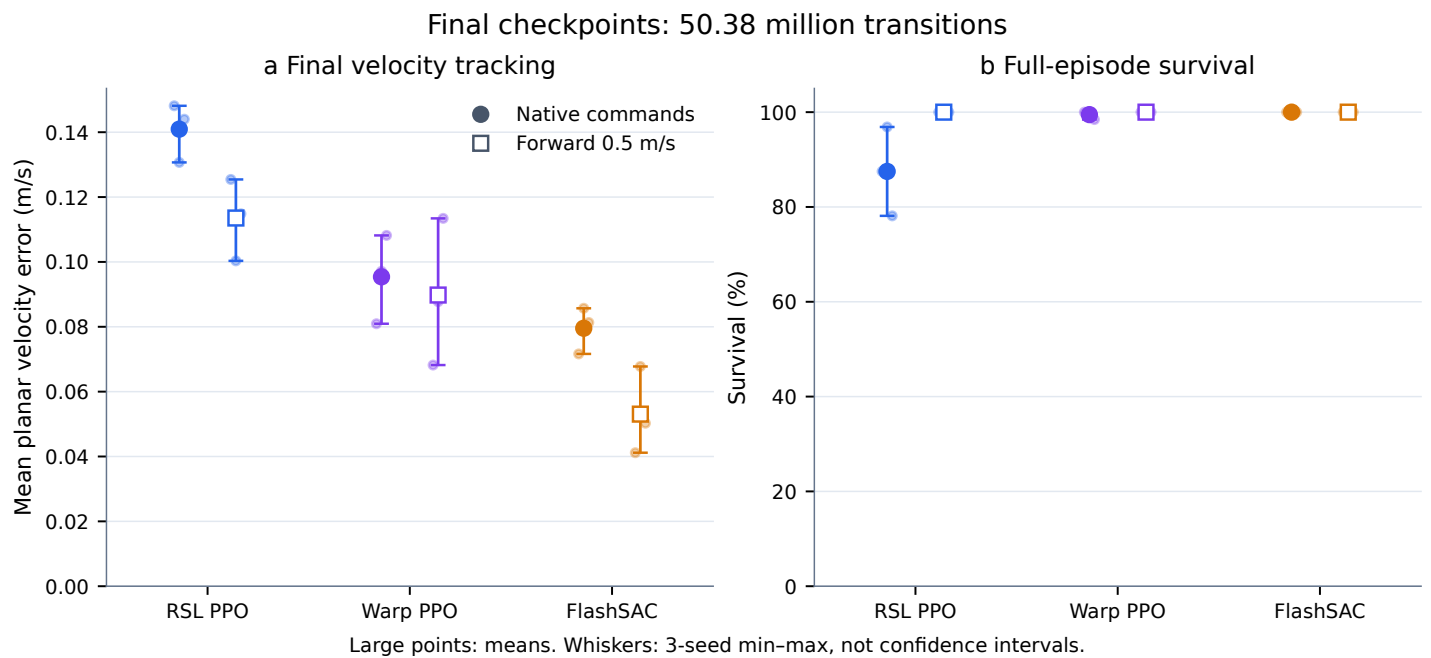


Figure 3. Final 50.38M-transition checkpoints, with 64 evaluation worlds per scenario and seed. Large points are means, faint points individual seeds, and whiskers three-seed min-max ranges, not confidence intervals. Filled circles show native commands including standing; open squares show forward 0.5 m/s. Survival measures completion of the native 20-second episode independently of velocity tracking.

Final forward evaluation at 0.5 m/s

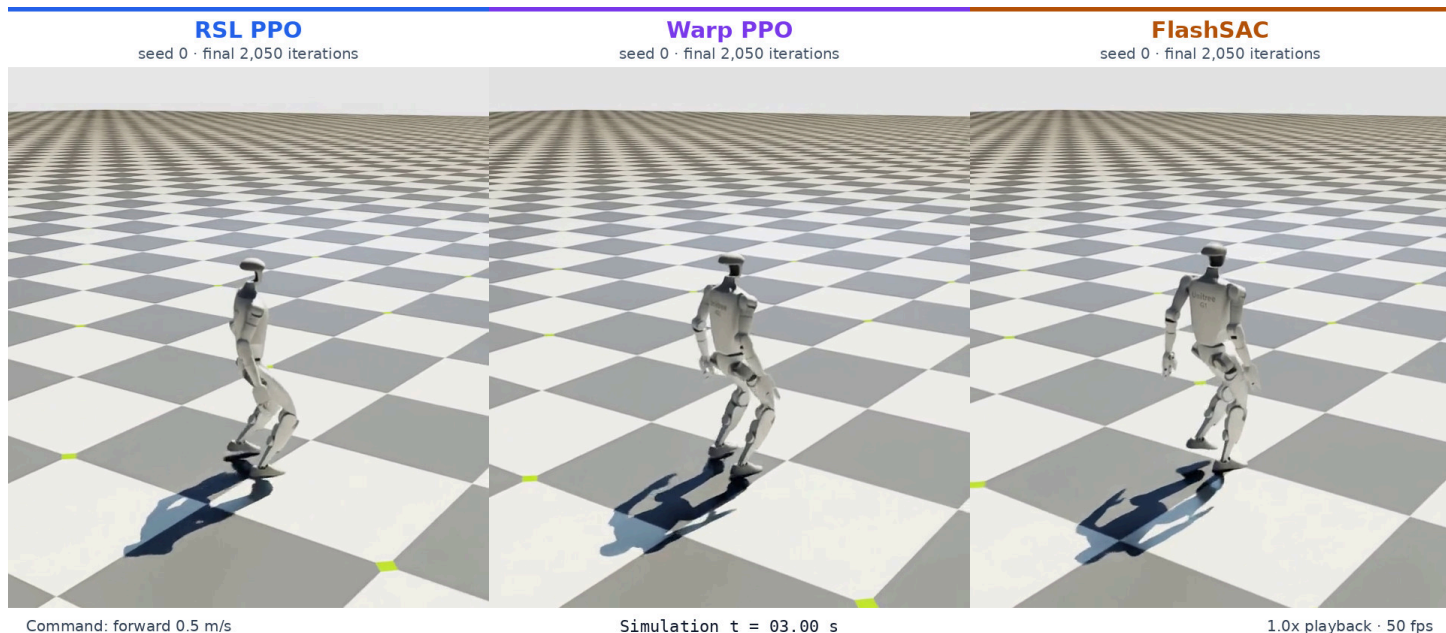
Learner	Mean planar error m/s ↓	Survival ↑	Walking success ↑
● RSL PPO	0.114 0.100–0.125	100.0%	100.0%
● Warp PPO	0.090 0.068–0.113	100.0%	100.0%
● FlashSAC	0.053 0.041–0.068	100.0%	100.0%

Small numbers below the error mean give the three-seed range. Survival and success percentages average seeds, each with 64 worlds and a maximum episode length of 20 seconds.

Recorded contact traces show repeated foot contacts, but FlashSAC’s left/right support duty is strongly asymmetric. These metrics establish commanded movement and survival. The replays below show one example per learner; further gait analysis is needed to establish balanced motion or suitability for hardware.

04 Final policies side by side

Replay the final checkpoint at iteration 2050 for learner seed 0 from each algorithm. All three policies receive the forward command of 0.5 m/s and use evaluation seed 10000. Seed 0 is selected consistently across learners; this is one illustrative replay per algorithm.



Video 1. Final G1 policies: RSL PPO, Warp PPO, and FlashSAC. Playback is synchronized for visual comparison; video playback speed does not measure training or simulation throughput.

The video always shows the forward scenario. The scenario selector above changes the plots and tables. Open an individual recording for a larger view on a small screen.

Side-by-side MP4 RSL PPO MP4 Warp PPO MP4 FlashSAC MP4

Recorded with Newton RTX on a local RTX 4090: 999 frames at 50 fps, or 19.98 simulated seconds at 1× playback speed. The camera follows world 0 using the same fixed viewpoint in each 64-world replay. Observation noise and pushes are disabled; native startup and reset randomization remain. The clips stop before the 20-second episode reset. Recording settings and checkpoint hashes are separate from the unchanged training measurements.

Scope of the recordings. These seed-0 replays help inspect movement. They do not replace the three-seed, 64-world evaluations, establish a natural or hardware-ready gait, or provide a new timing measurement.

05 Experimental setup

Environment

Isaac-Velocity-Flat-G1, Isaac Lab 3 on Newton MJWarp. Shared rewards, observations, commands, action scale, resets, terminations, and initial episode-counter randomization. All nine runs have the same recorded MDP hash.

Common evaluation

64 worlds, 20-second native episodes, deterministic policy actions, noise and pushes disabled.
Evaluation seed = 10000 + learner seed.
Checkpoints: 250, 500, 1000, 1500, 2050. Two scenarios produce 90 evaluations total.

Training budget and hardware

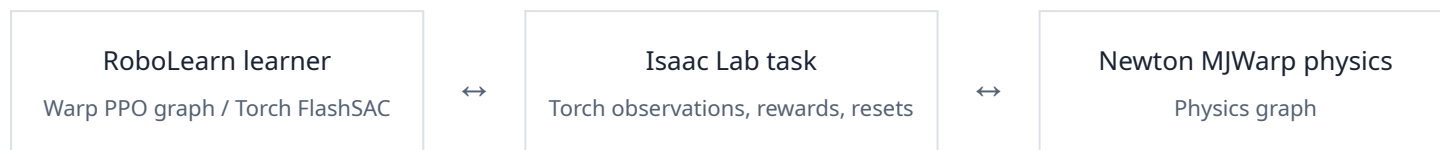
1,024 parallel environments × 24 steps × 2,050 iterations = 50,380,800 collected transitions. Fresh policy and optimizer for seeds 0, 1, 2. Each run had one isolated L40, 15 CPUs, and 64 GiB RAM.

Walking gate

At least 80% of worlds satisfy walking, mean planar error ≤ 0.2 m/s, and survival $\geq 90\%$. A walking world survives with planar error < 0.25 m/s, yaw-rate error < 0.4 rad/s, and forward speed > 0.25 m/s.

Measured software. Python 3.12.14; Torch 2.12.0+cu130 / CUDA 13.0; RSL-RL 5.5.1; Warp 1.17.0; Warp-NN 0.4.0; Newton 1.6.1; MuJoCo Warp 3.12.0. The measured RoboLearn distribution was 0.1.0 at the immutable revision listed below.

Capture scope



The MDP runs in Torch. Warp PPO's learner graph is separate from the physics graph; this experiment does not capture a complete physics-MDP-learning iteration in one Warp graph.

► Algorithm settings and learning work

► Use the current integration or reproduce the frozen experiment

06 Interpretation and limits

- **A useful result, with a narrow scope.** Three learner seeds on one flat G1 task support the observed sample-efficiency and runtime conclusions. They do not establish universal speed or quality advantages. Matching seed numbers does not match every implementation's random draw.
- **Shared MDP, algorithm-specific work.** Flash uses bounded actions, a larger model, distributional critics, replay, and AMP. PPO uses unclipped Gaussian actions and FP32 updates. Matching collected transitions does not match optimizer work or policy support.
- **Sampled success is not an exact convergence time.** Evaluations were performed at five checkpoints after training. First-passing times are retrospective checkpoint production times; regressions and unseen behavior between checkpoints matter.
- **Integration benchmark, not a paper replication.** The FlashSAC paper used Isaac Lab 2.1 with PhysX and a different PPO budget. This benchmark uses Isaac Lab 3/Newton, the authors' code recipe, and an equal collected-transition budget. The paper's separate real-robot experiment is also a different setup.
- **Current graphs leave rollout costs.** A captured learner does not remove Torch MDP work or guarantee faster matrix operations. A fully captured Warp MDP and physics-learning graph would require a separate implementation and comparison.

07 Evidence and reproducibility

All nine bundles passed SHA256 verification, checkpoint and optimizer audits, budget checks, and common-evaluation checks. Weights changed and remained finite. Native checkpoints are preserved; raw measurement JSON is unchanged. The three earlier Flash runs with different initial episode counters are retained as diagnostics and excluded here.

Measurements refer to the source revisions below. RoboLearn 0.2 preserves those learner kernels and math, and fixes the adapter's unclipped-action storage with one device conversion/copy. That fix was made after these runs; no new timing measurement is claimed for it.

The public JSON and CSV downloads are compact numeric extracts of the verified archive. They retain full-precision evaluations, timing, update counts, and gait aggregates. Original raw JSON, contact traces, native checkpoints, and execution logs remain in the experiment archive. The table hashes identify the original sources; hashes of these published files are recorded in SHA256SUMS. The provenance file links each extract to its original-source hash.

► **Source revisions and immutable measurement hashes**

References and attribution

1. Donghu Kim, Youngdo Lee, and coauthors. FlashSAC, 2026. Official implementation at the adapted revision. RoboLearn retains the authors' copyright and MIT license.
2. John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms, 2017.
3. RSL-RL, Isaac Lab, Newton, and MuJoCo Warp: their respective authors and contributors.
4. NVIDIA and contributors. Warp-NN and Warp. RoboLearn's Warp PPO is an independent implementation of the published PPO objective.
5. RoboLearn. Full attribution and BibTeX citations.